

Robot Dynamics & Control

2025

Session 2 – Serial Robot Modelling and Statics

Tutors:

Pedro Luis Figueroa Saire

Danial Sabzevari

Emails: name.surname@edu.unige.it (or Teams)

Dibris



UNIVERSITÀ DEGLI STUDI
DI GENOVA

Outline

- **Serial robot modelling** – how to link rigid bodies and build a 2R robot from scratch
- **Manipulator statics** – how to derive gravity compensation torques for the model

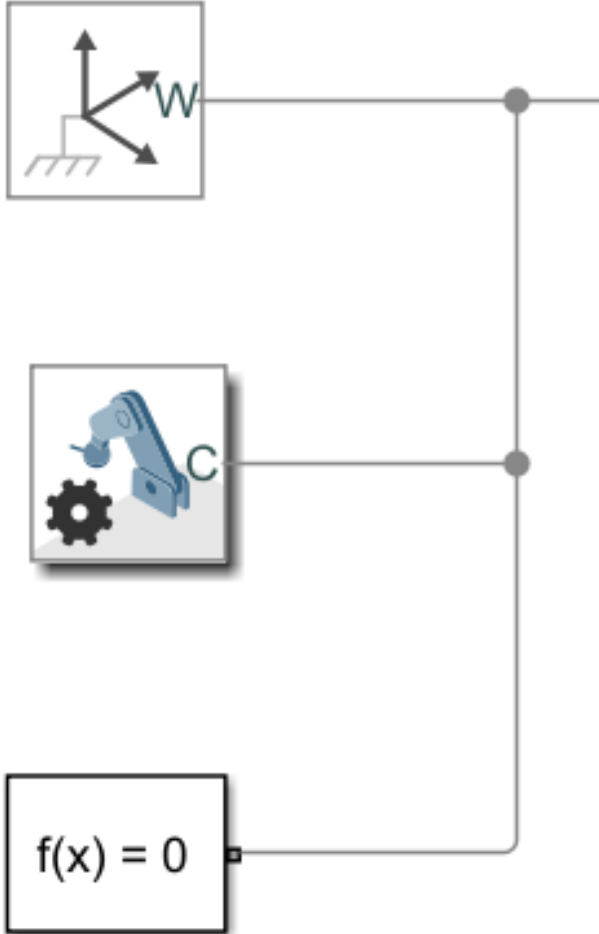


Part I

Serial Robot Modeling



Blank Simscape Mechanism (recap)



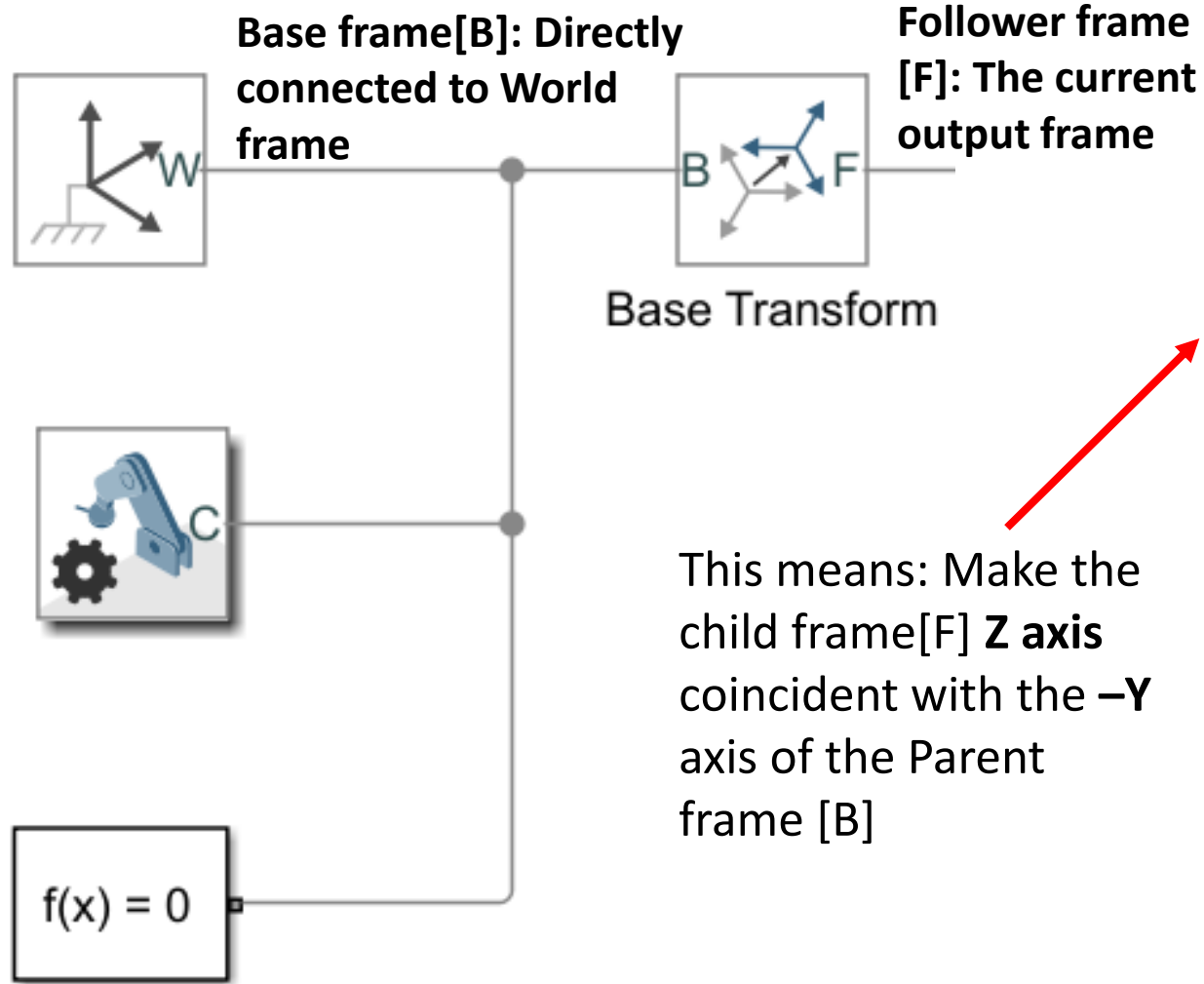
World Frame: any other rigid body will be referred to this coordinate system

Mechanism Configuration: Set gravity (**Hint:** when debugging robot's motion it can be useful to **turn it off**, writing the $[0\ 0\ 0]$ vector).

Solver Configuration: for our purpose default settings are fine

These blocks are essential to build
any kind of multibody system

Base rigid transform



This means: Make the child frame[F] **Z axis** coincident with the **-Y** axis of the Parent frame [B]

Block Parameters: Base Transform

Rigid Transform

Settings Description

NAME	VALUE
✓ Rotation	
Method	Aligned Axes
✓ Pair 1	
Follower	+Z
Base	-Y
✓ Pair 2	
Follower	+X
Base	+X
✓ Translation	
Method	None

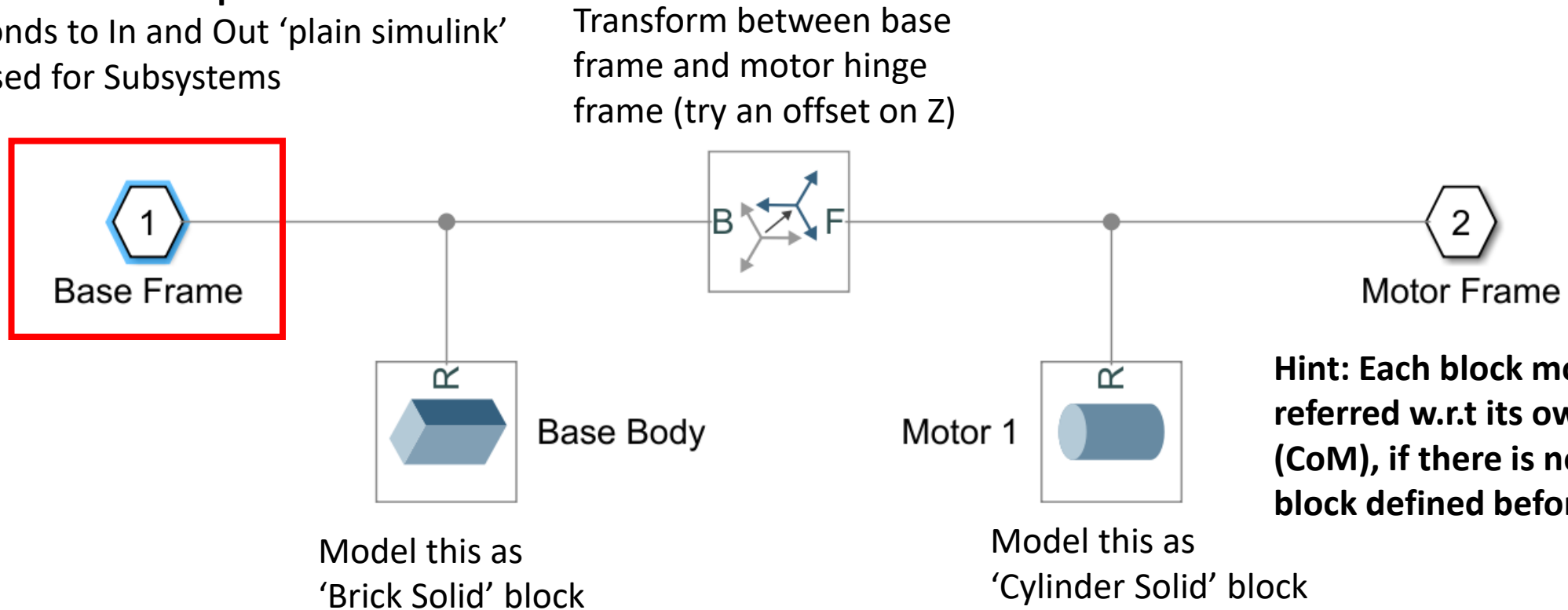
It's not mandatory, but you can rotate the world frame so that model building is more intuitive



Base Link

This is the **'connection port'** block.
Corresponds to In and Out 'plain simulink'
blocks used for Subsystems

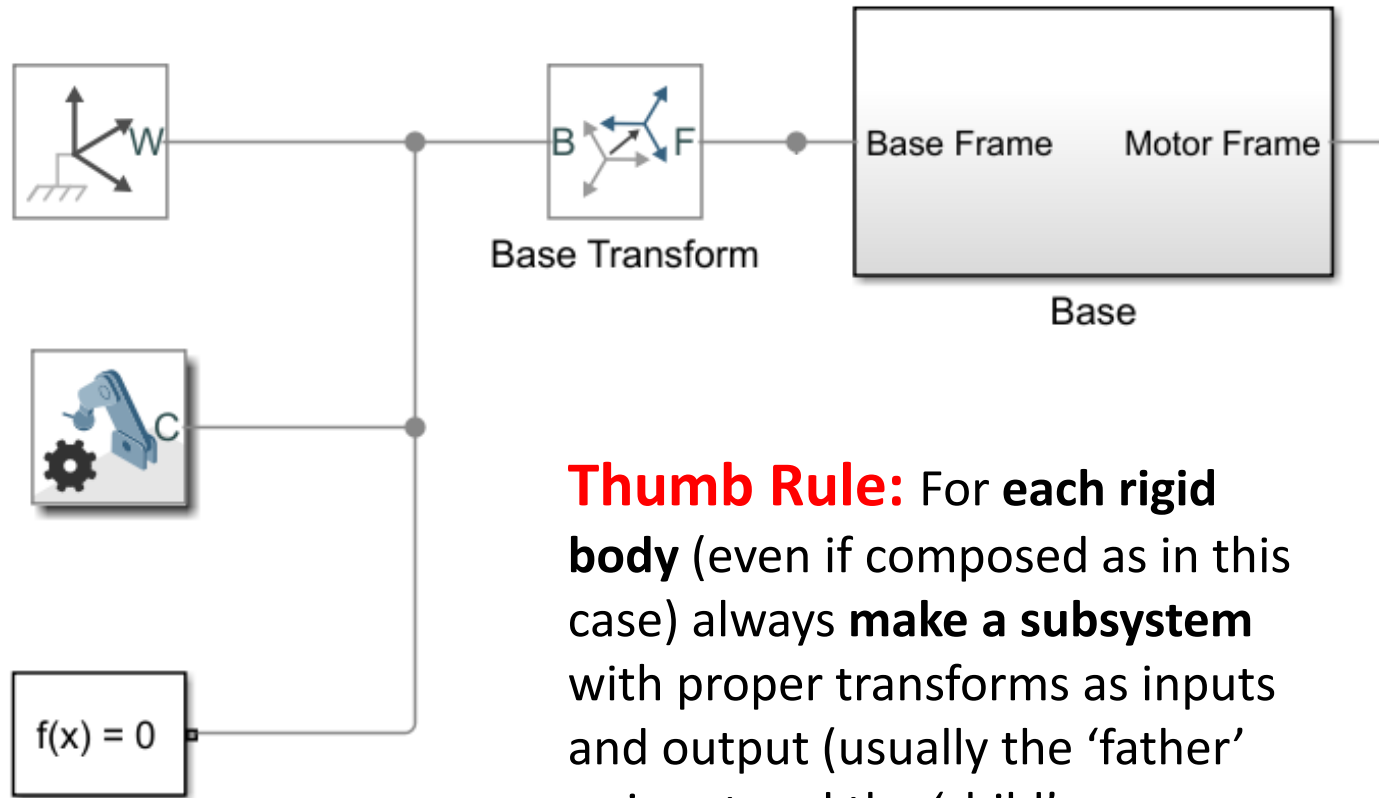
Goal: Create a base, composed of two
bodies [Base body and Motor 1]. Base
body is defined with respect to (w.r.t)
world frame.



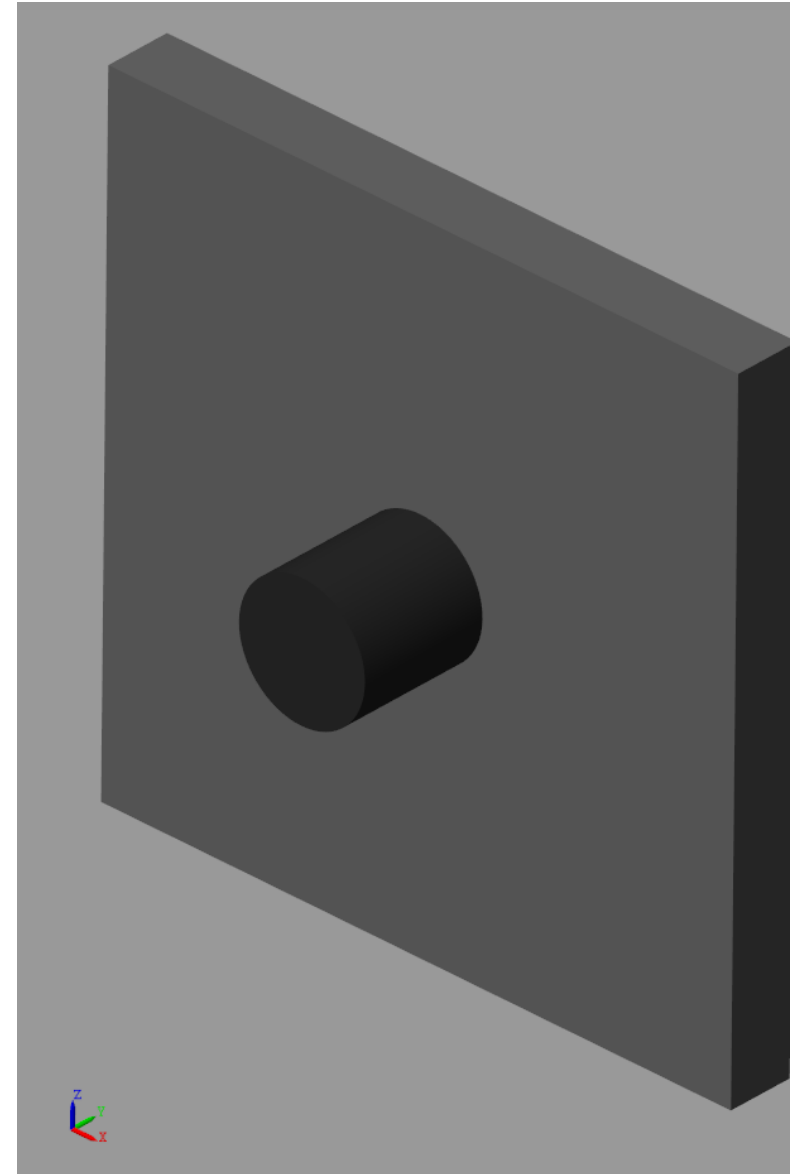
Here it's mainly a 'visual aid' but in practice
every (non-mobile!) manipulator has a **fixed
base to refer the other links to.**



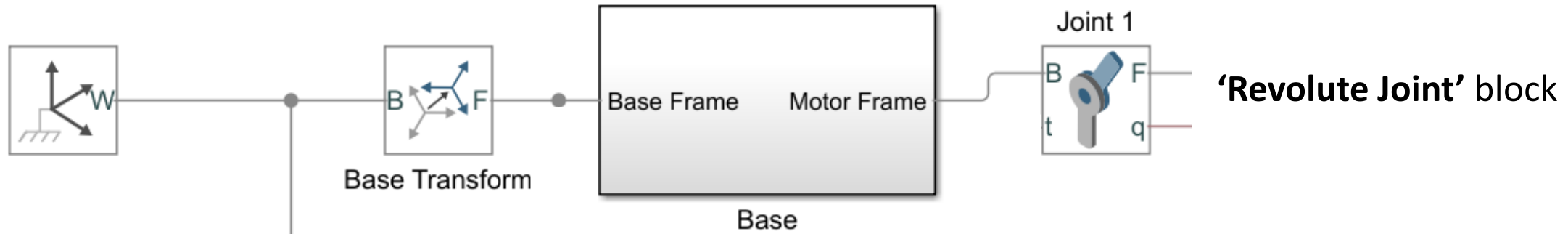
Base link



Thumb Rule: For each rigid **body** (even if composed as in this case) always **make a subsystem** with proper transforms as inputs and output (usually the ‘father’ as input and the ‘child’ as output).



Simple pendulum – Add a Joint



Block Parameters: Joint 1

Revolute Joint ☒ Auto Apply ?

Settings Description

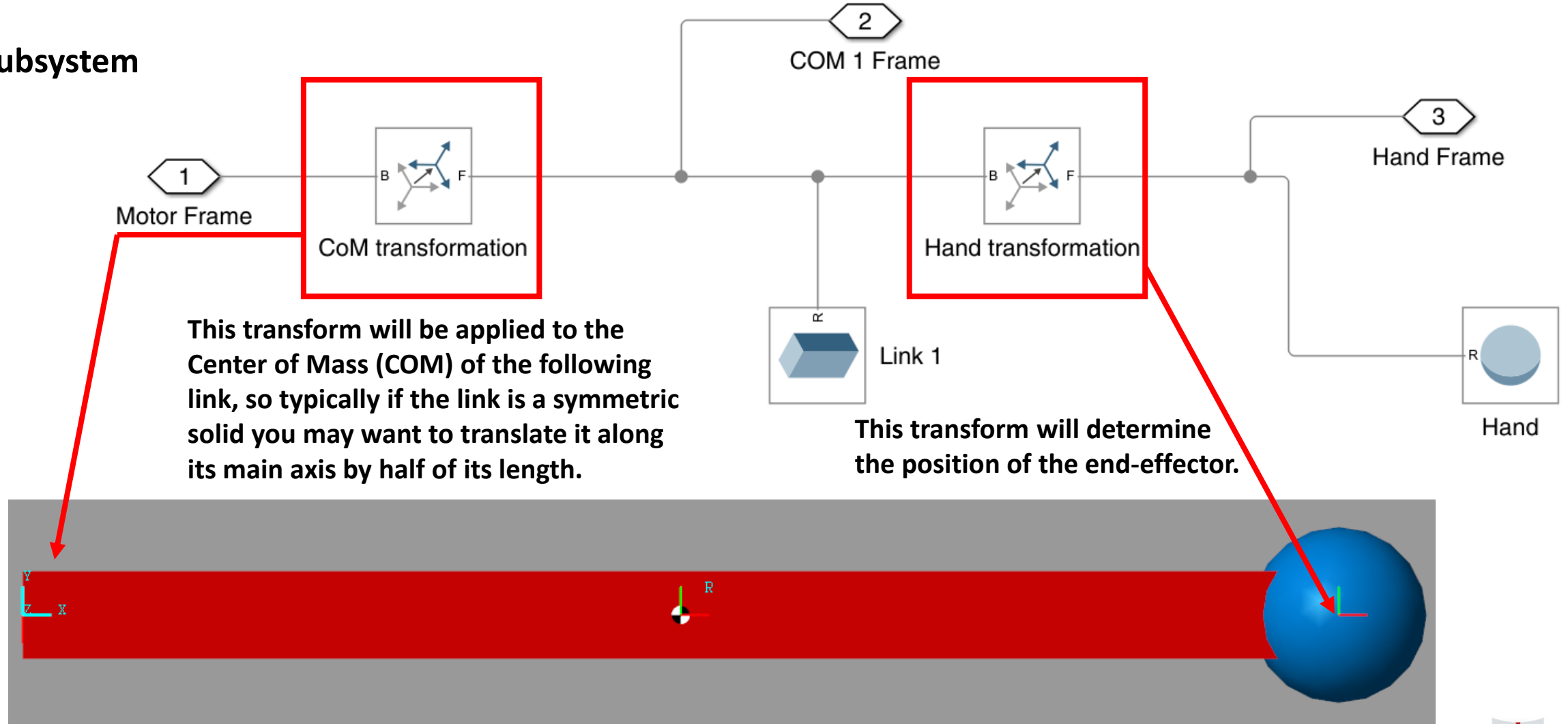
NAME	VALUE
▼ Z Revolute Primitive (Rz)	
> State Targets	
▼ Internal Mechanics	
Equilibrium Position	0 rad Compile-time ▼
Spring Stiffness	0 N*m/rad Compile-time ▼
Damping Coefficient	0.002 N*m/(rad/s) Compile-time ▼
> Limits	
▼ Actuation	
Torque	Provided by Input ▼
Motion	Automatically Computed ▼
▼ Sensing	
☒ Position	

Hint: When modeling physical systems, always consider **damping effect**. Joints are mechanisms, and common mechanisms have always some **friction due to moving parts**. **Without damping a simulated joint** under gravity will exhibit **undamped oscillations**.



Simple pendulum – Add a link

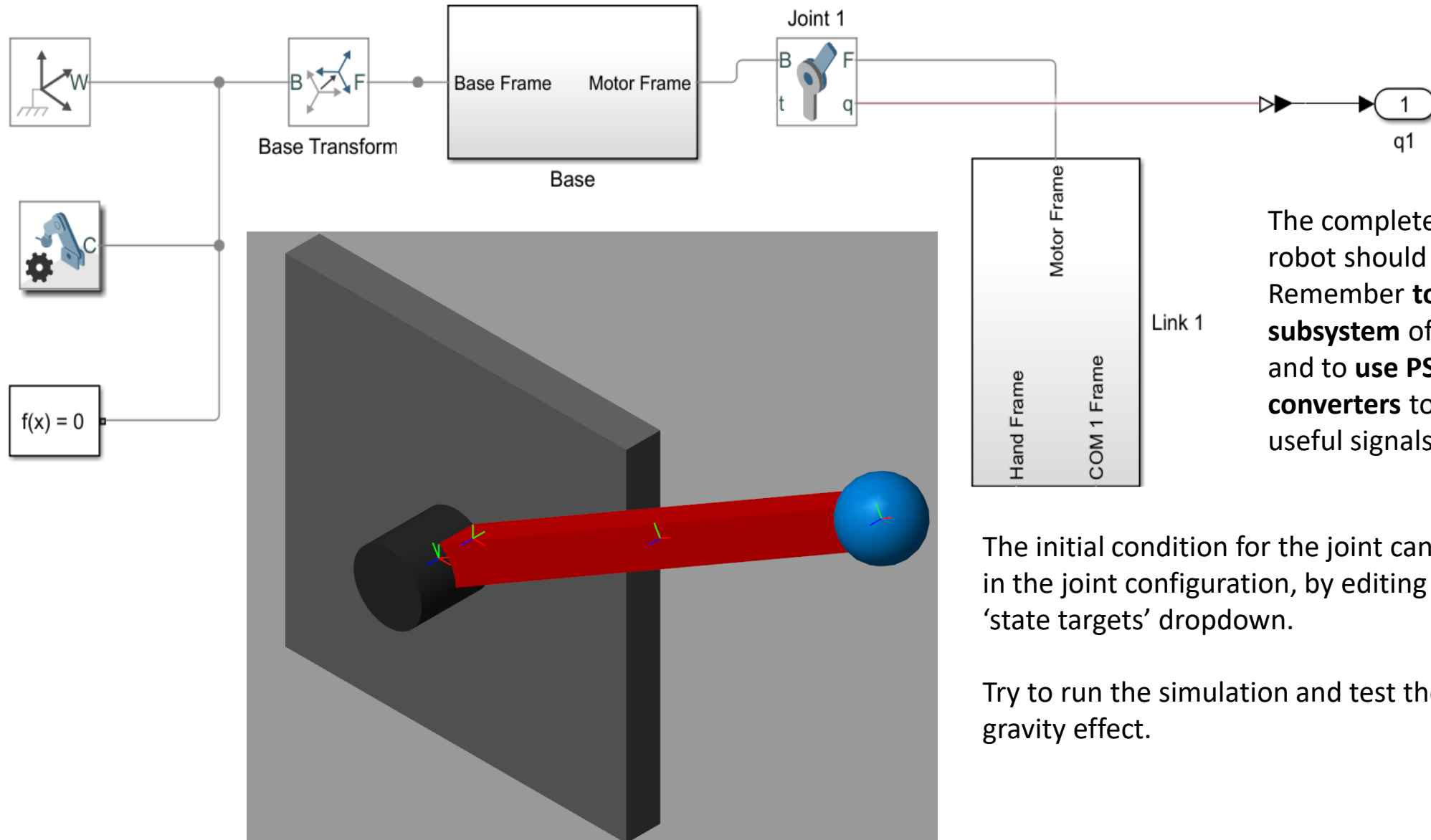
Link Subsystem



Remember to toggle frames and centers of mass in the mechanics explorer



Simple pendulum – 1R Robot



The complete model of a 1R robot should look like this. Remember **to make a subsystem** of the mechanism and to **use PS-Simulink converters** to bring out useful signals.

The initial condition for the joint can be set in the joint configuration, by editing the 'state targets' dropdown.

Try to run the simulation and test the gravity effect.

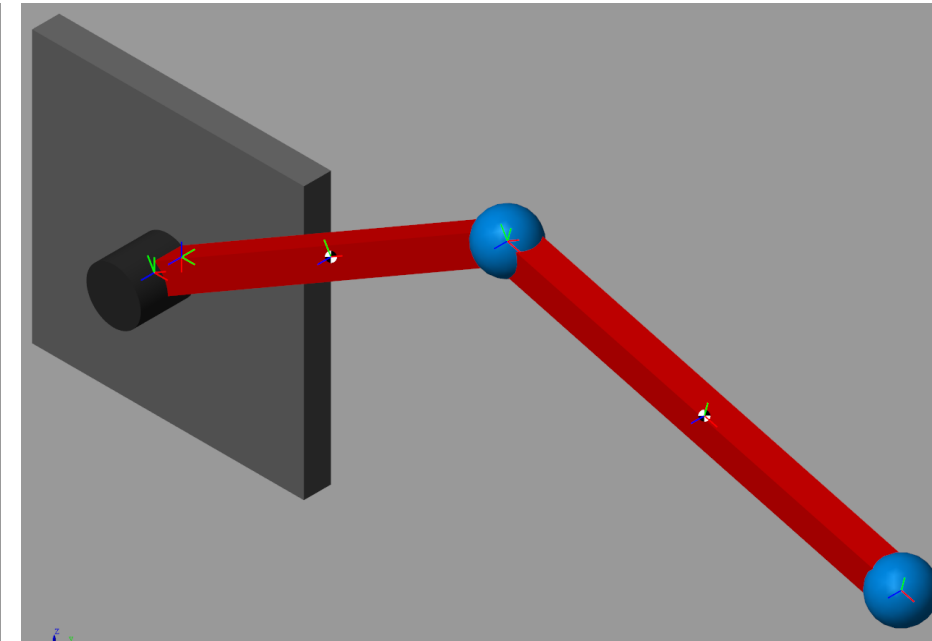
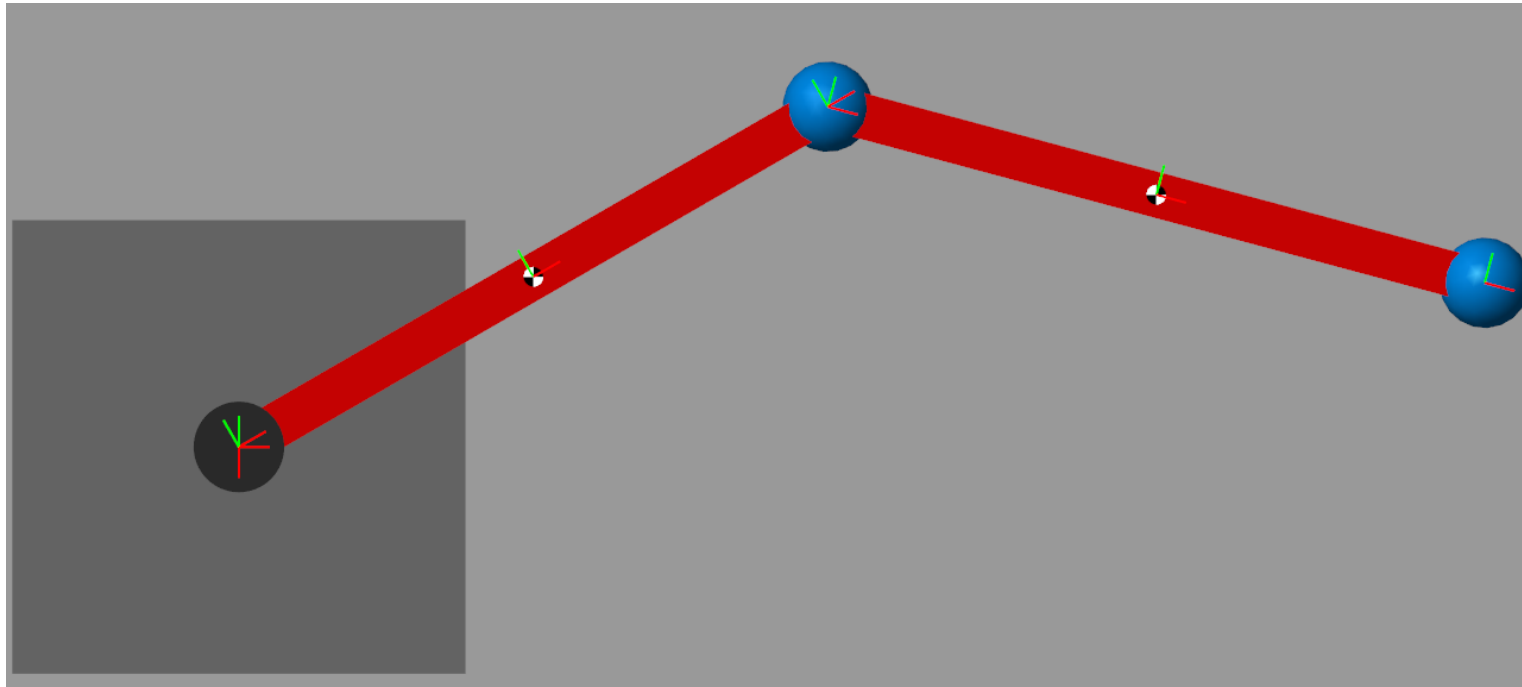


2R Robot Model

The **1R robot design** can be iterated adding further joints and rigid links.

Always remember **to refer each transform to the previous one** in the serial chain, so **each body** can be **placed relatively to the previous**.

Exercise: build a 2R robot (or double pendulum)



Part II

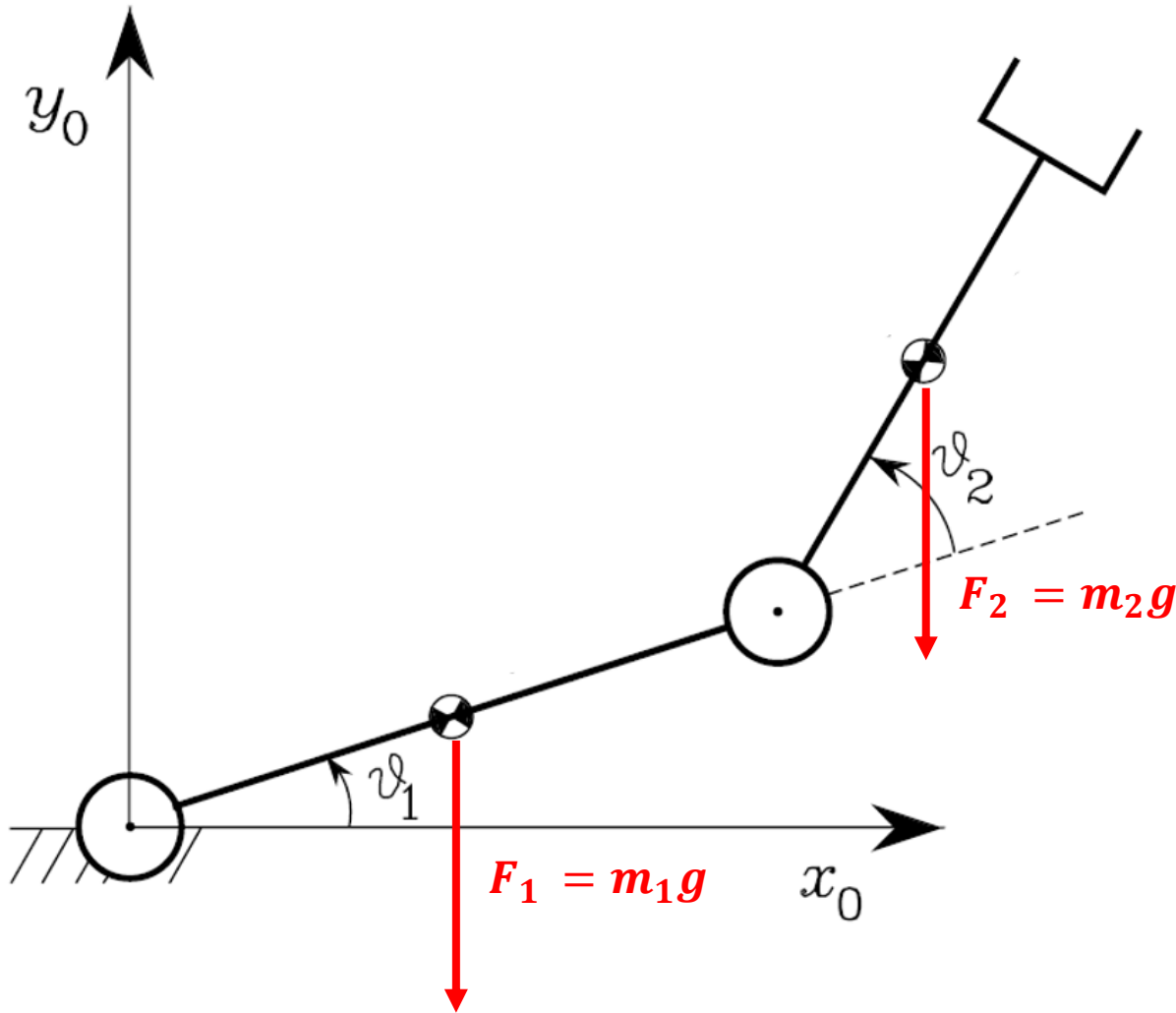
Statics:

Gravity Torque

Compensation



The *gravity* action



Gravity **always** acts on any rigid body on earth

Here we assume $\mathbf{g} = [0, -9.81]$

We have $\mathbf{F}_1$ acting on link 1 and $\mathbf{F}_2$ acting on link 2

But we control robot's motion through its joints, so **how does this reflect on joints?**

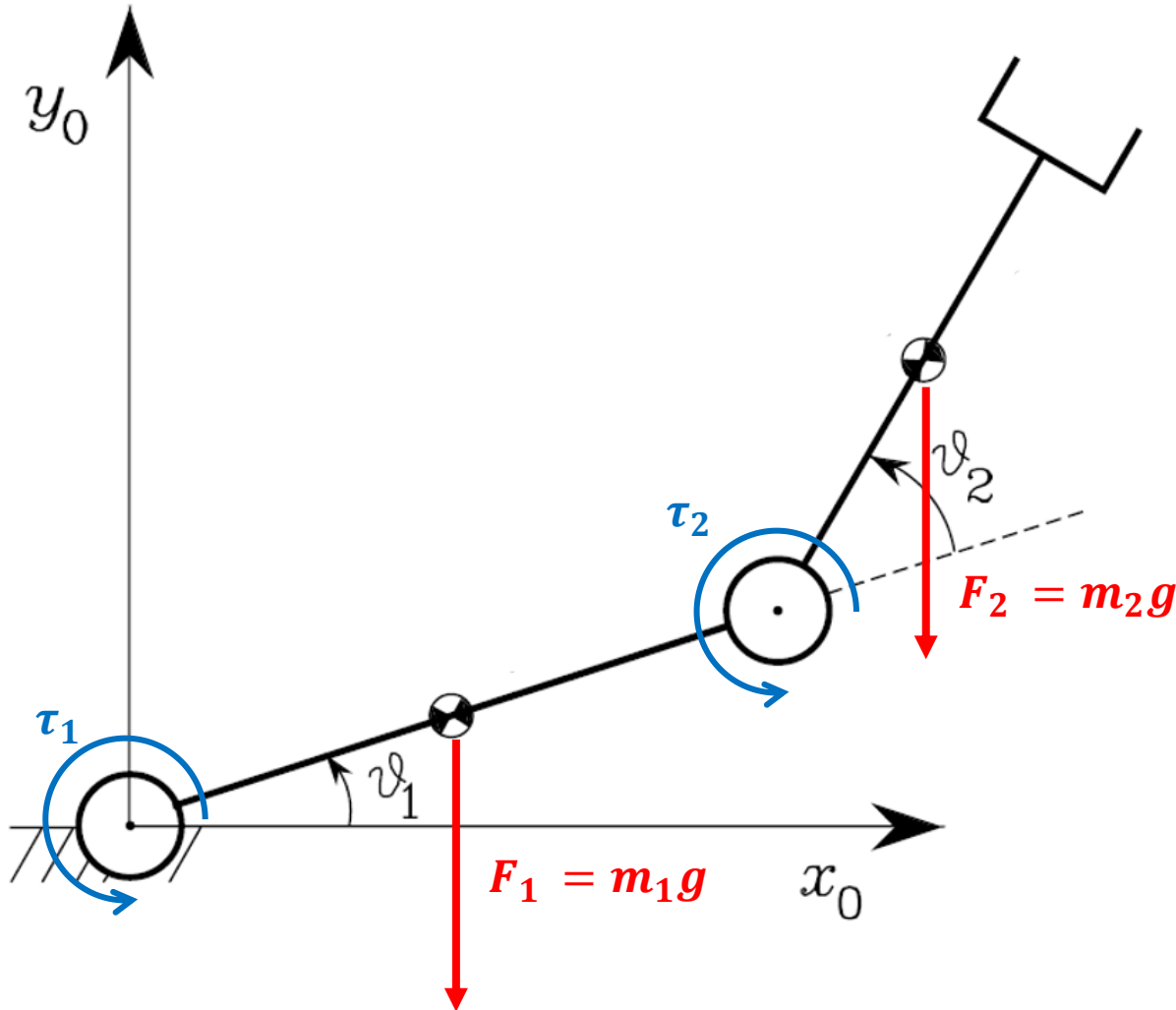
The *gravity* joint torques

From the principle of **virtual work**, we obtain the following:

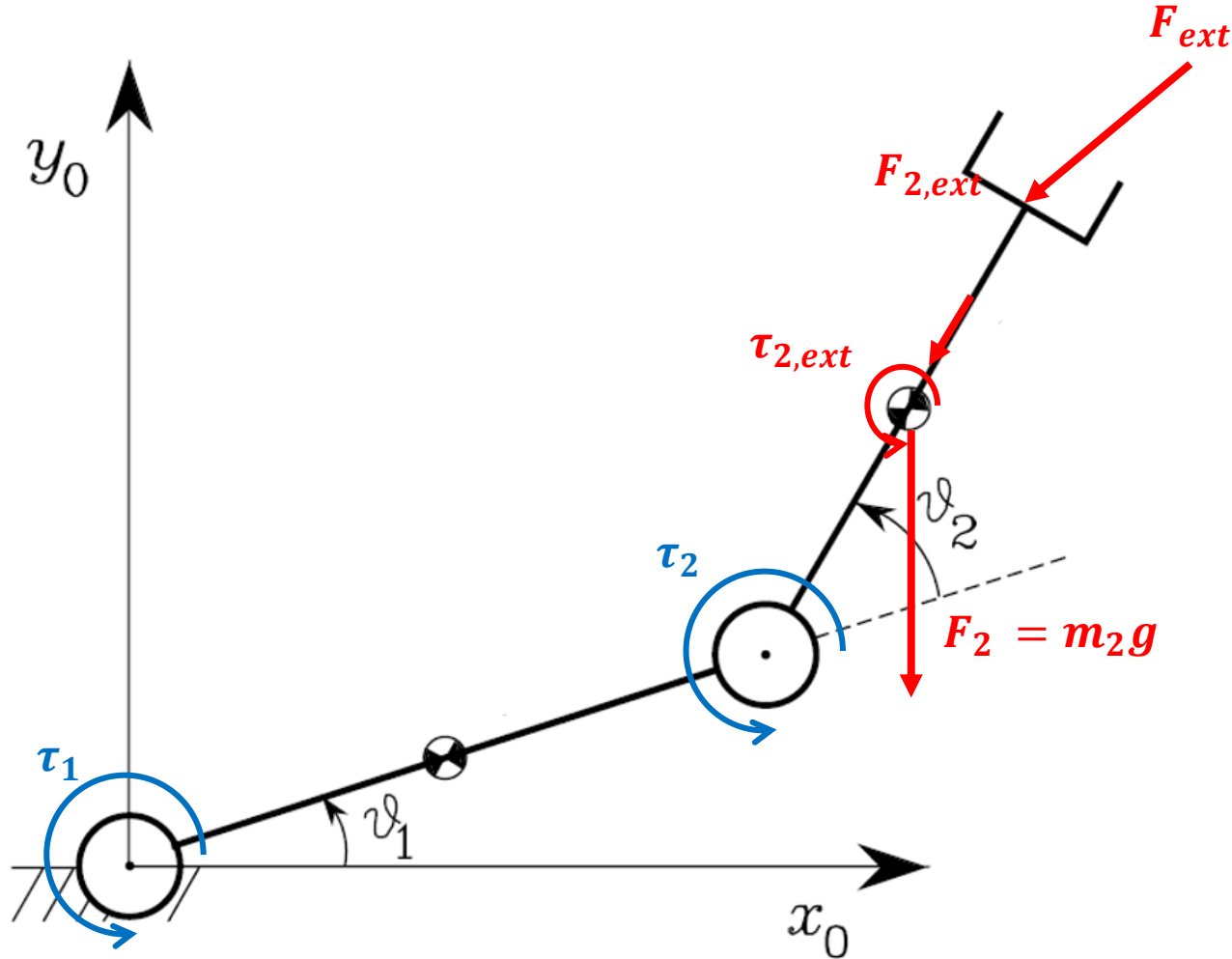
$$\boldsymbol{\tau} = \mathbf{J}^T \mathbf{F}$$

It can be used to **specify a task** in terms of **cartesian forces and torques** and compute **how it affects joint torques**

In this case torques can be used to **cancel out gravity effects!**



An *external force* at the end effector

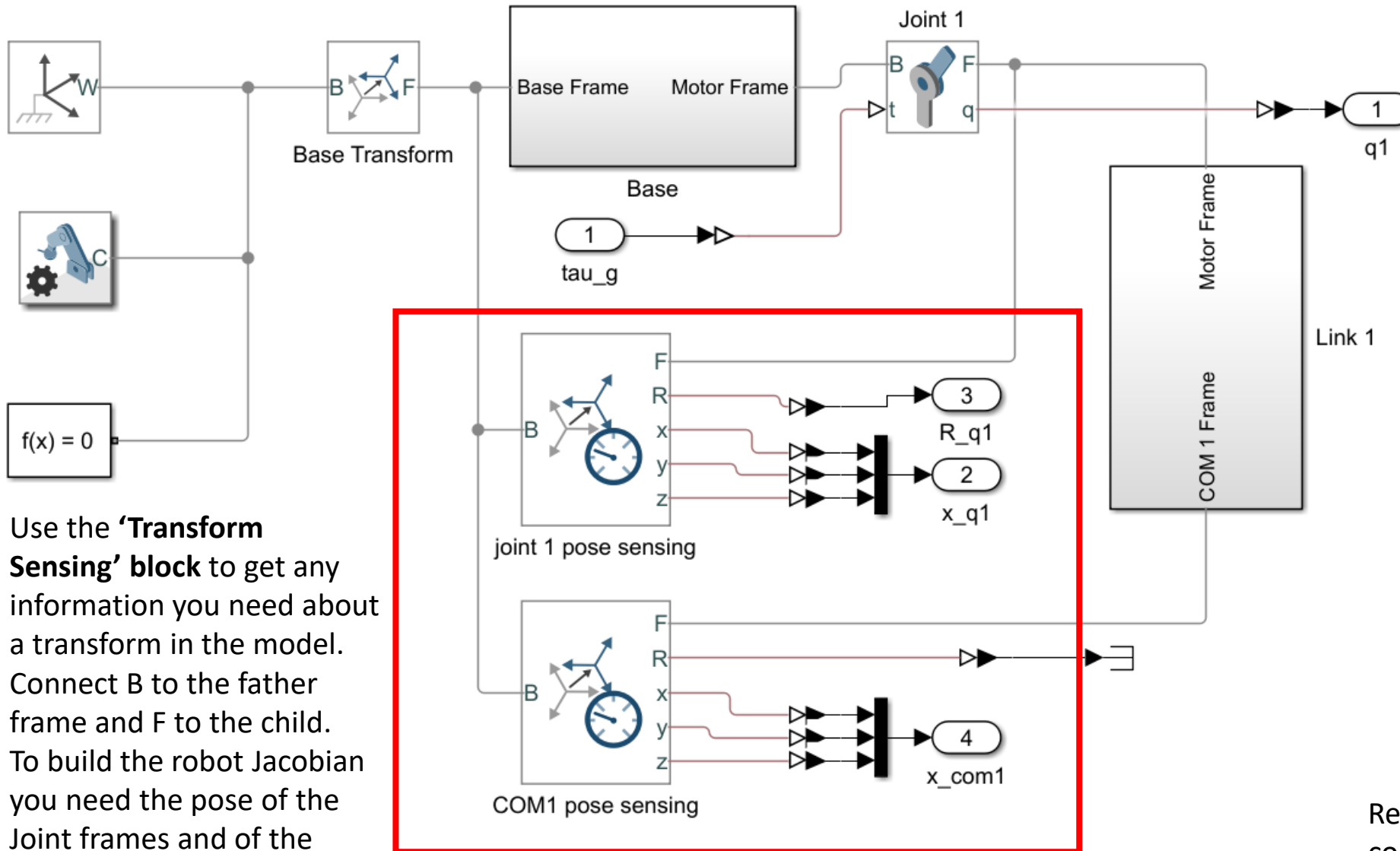


If we only consider the effect of F_{ext} :

$$\tau = J_{ee}^T F_{ext}$$

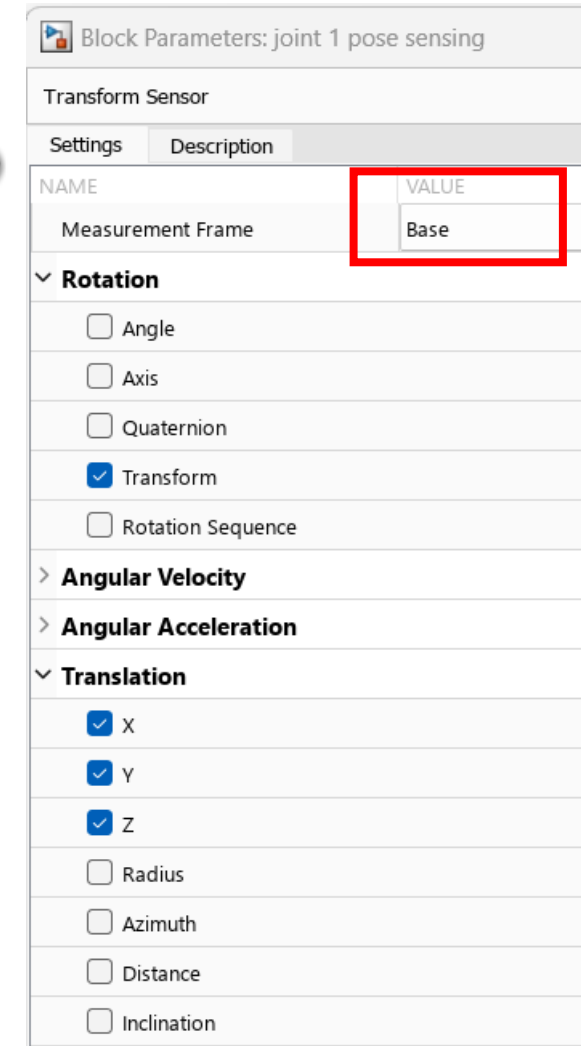
If we need to consider other forces acting on COM 2, we compute the effects of F_{ext} on it, so both linear and angular Jacobians will be needed because a torque will be generated about the COM.

Gravity Compensation



Use the **'Transform Sensing' block** to get any information you need about a transform in the model. Connect B to the father frame and F to the child. To build the robot Jacobian you need the pose of the Joint frames and of the target point (COM in this case)

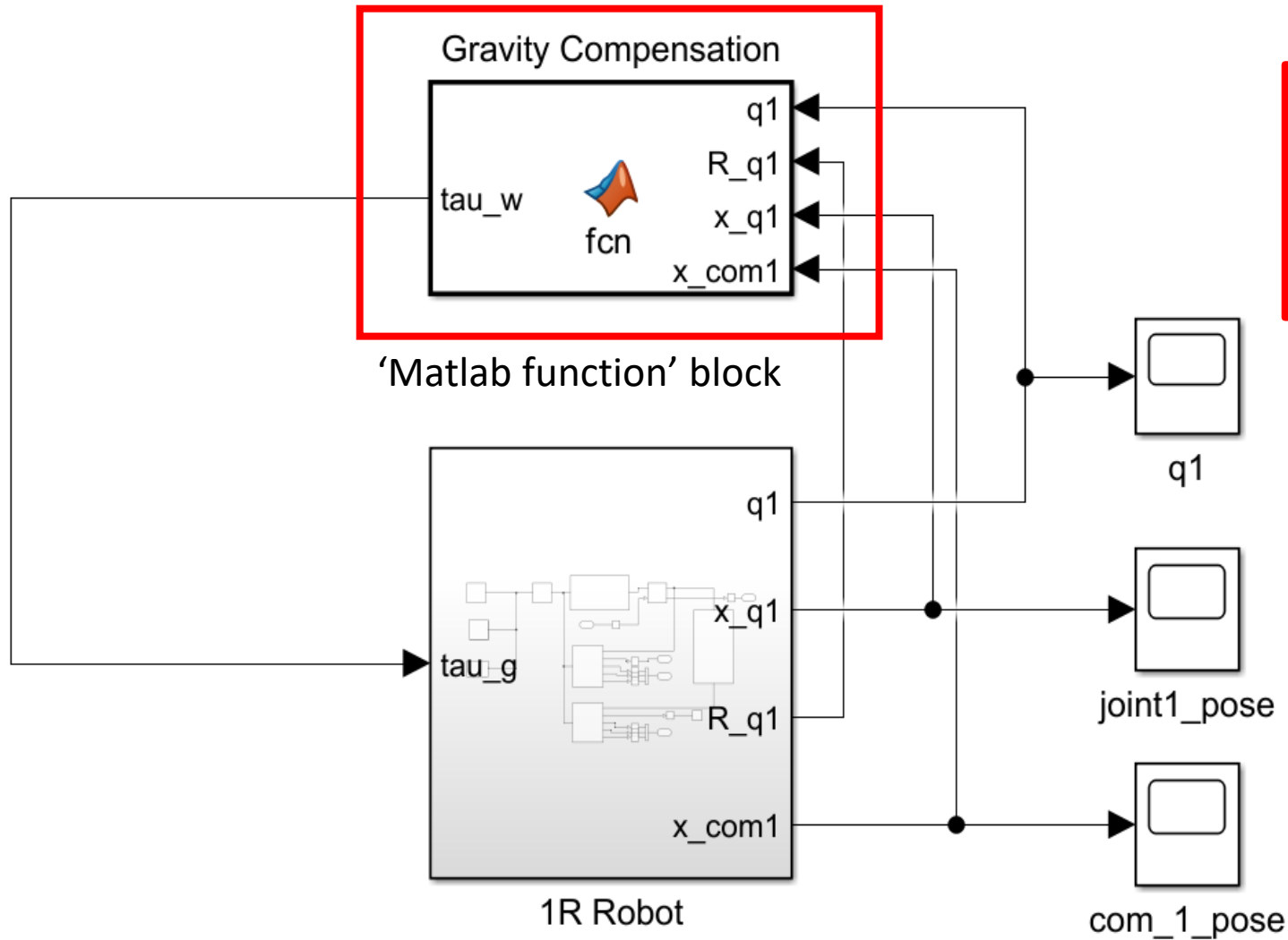
Hint: Use multiplexer blocks to merge translation signals into vectors.



Remember to choose coherent reference frames



Algorithm Implementation



```
function tau = fcn(q1)
% ----- useful quantities (gravity, link length, link mass)
% ----- compute jacobian of the COM
% ----- get static torque from balancing equation
```

Exercise: Implement gravity compensation on 1R and 2R robots.

Hint: There are multiple ways to build Jacobian at each control step. Try to use the Geometric Jacobian.

